

RAPPORT DE PROJET DE MASTER

R.A.I.D.

RAM Artificial Inspection Dump

1^{er} juin 2026

HENRY Amand — ROUSÉE Tom — SICOT Théo

Supervisé par :

GIGUET Emmanuel — BENOIT Tristan

Résumé

Ce projet présente le développement de R.A.I.D. (RAM Artificial Inspection Dump), une solution d'analyse forensique visant à classifier les données issues de la mémoire vive (RAM). L'objectif central est de segmenter des dumps mémoire bruts pour distinguer les données chiffrées des informations non chiffrées (textes, images, PDF) sans connaissance préalable de leur structure. L'approche repose sur l'utilisation d'un modèle d'apprentissage profond de type *Transformer*, capable d'évaluer la prédictibilité des séquences d'octets. La méthodologie comprend la génération d'un jeu de données synthétique, l'entraînement d'un classifieur d'octets avec mécanisme d'attention, et une validation par vote pondéré.

Les résultats démontrent que l'approche s'affranchit des vulnérabilités des méthodes traditionnelles basées sur des signatures statiques, prouvant la viabilité des architectures Transformer pour l'interprétation sémantique de flux binaires bruts. Grâce à la stratégie de vote pondéré, le système parvient à transformer des prédictions locales incertaines en une segmentation globale robuste. Si le modèle isole avec succès les zones chiffrées des structures conventionnelles, la distinction mathématique entre le chiffrement et la forte compression à haute entropie demeure un défi complexe.

[NEED TO CHANGE] : Change le résumé

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Problématique	4
1.3	Objectifs	5
2	Méthodologie	6
2.1	Générateur de datasets synthétiques	6
2.1.1	Collecte de données	6
2.1.2	Transformation des données	6
2.1.3	Création du segment RAM synthétique	7
2.1.4	Construction des différents "dumps"	8
2.2	Le découpage en chunks	9
2.3	Modèle de classification	9
2.3.1	Introduction au principe du <i>Transformer</i>	9
2.4	BytesTransformerClassifier	12
2.5	Entraînement et validation	14
2.6	Inference	15
2.6.1	Processus d'inférence	15
2.6.2	Principe de <i>l'overlapping</i> (recouvrement)	15
2.7	Optimisation	16
2.8	Outils d'analyse	18
3	Résultats	21
3.1	Le vote pondéré	21
3.2	Comparaison des configurations du modèle	21
3.3	Performance Globales	22
3.3.1	Matrices de confusion	22
3.3.2	Métriques d'évaluation	24
4	Discussion	25
4.1	Performance du vote pondéré	25
4.2	Chiffrement et compression : l'entropie trompe le modèle	25
4.2.1	La convergence statistique des hautes entropies	26

4.2.2	Impact sur la précision de la segmentation	26
4.2.3	Limites de l'approche sémantique pure	26
4.2.4	Sémantique binaire vs Identification rigide	26
4.2.5	Résilience face à l'obfuscation	27
4.2.6	Le coût de l'intelligence	27
4.3	Limites	27
4.3.1	La généralisation face aux données réelles	27
4.3.2	Le choix de l'architecture <i>Transformer</i>	28
4.4	Perspectives	29
4.4.1	Passage en apprentissage non supervisé	29
4.4.2	Le défi de la fragmentation et de la reconstruction	29
5	Conclusion	31
	Bibliographie	32

1 Introduction

1.1 CONTEXTE

L'analyse forensique de la mémoire vive (RAM) constitue un pilier fondamental de l'informatique légale. Contrairement au stockage persistant, la RAM contient des artéfacts volatils critiques : clés de chiffrement, fragments de documents, ou résidus de textes souvent invisibilisés dans de grandes quantités de données déstructurées.

Jusqu'au années 2000, aucun outils n'étaient conçu pour l'analyse, les experts se reposaient sur des méthodes ad hoc, et donc utilisaient des outils générique comme `strings` ou `grep` pour récupérer les chaines de caractères. Par la suite, pendant la premiere décennie du siècle, des outils comme *Volatility* ou *Rekall* ont émergé, offrant des capacités d'extraction plus avancées basées sur des signatures statiques.

Cependant, l'extraction d'informations reste complexe. Bien que ces outils aient progressé, les méthodes traditionnelles reposent souvent sur des signatures statiques, qui sont à la fois rigides et vulnérables à l'obfuscation. De plus, la nature fragmentée des dumps mémoire rend difficile l'identification de structures de données critiques, sans compter la présence de données chiffrées ou compressées qui présentent une entropie élevée et peu de motifs exploitables.

Dans ce contexte, l'approche par apprentissage automatique, afin de trouver des règles sémantiques dans les flux binaires, offre une alternative prometteuse. Il y a quelque années, le travail de Thomas Gougeon « *Analyse forensique de la mémoire des cartes à puce* » ont démontré la capacité d'un modèle de langage à différencier les données chiffrées des données textuelles dans le contexte spécifique des cartes à puce. Cependant, le passage à l'échelle pour traiter des dumps mémoire complets, avec leur diversité de formats et de structures, reste un défi ouvert.

1.2 PROBLÉMATIQUE

[NEED TO CHANGE] : Reformuler la problématique?

Comment segmenter de manière fiable et autonome un dump de RAM, en distinguant les données chiffrées, des données non chiffrées, y compris des données complexes telles que des PDF ou des données compressées ?

La complexité de la tâche réside dans la similitude mathématique entre les données

chiffrées et compressées, toutes deux à haute entropie. De plus, la fragmentation, se qui créer une discontinuité des les données, rend l'identification de structures de données plus conventionnelles (textes, images) plus difficile.

1.3 OBJECTIFS

Le projet R.A.I.D. (RAM Artificial Inspection Dump) s'inscrit dans cette dynamique en proposant une approche basée sur l'apprentissage profond. L'objectif central est de développer un système capable de classer les octets d'un dump de RAM pour déterminer leur nature : chiffrée ou non, et ce de manière autonome.

L'implémentation repose sur l'utilisation de l'architecture *Transformer* et en particulier l'encodeur ainsi que son mécanisme de *self-attention* associé.

L'hypothèse de travail est qu'un modèle de langage, entraîné à classer des séquences d'octets, sera capable de différencier les données chiffrées, des données compressées, par nature à forte entropie, tout en identifiant facilement les structures de données plus conventionnelles et prévisibles, telles que du texte brut.

Cette différenciation doit permettre à terme de segmenter précisément la mémoire et d'identifier les différents champs de données sans supervision humaine.

2 Méthodologie

2.1 GÉNÉRATEUR DE DATASETS SYNTHÉTIQUES

2.1.1 Collecte de données

Pour constituer notre corpus initial, nous avons agrégé des données issues de trois sources principales :

- des textes bruts provenant du *projet Gutenberg*
- des documents PDF via *arXiv*
- des images via *Lorem Picsum*

Le *projet Gutenberg* est une bibliothèque numérique de plus de 60 000 livres du domaine public, offrant une grande diversité de textes littéraires. *ArXiv*, quant à lui, fournit un accès à des milliers de documents scientifiques au format PDF. Et enfin, *Lorem Picsum* propose une vaste collection d'images. Les trois services offrent des API permettant de récupérer facilement des données de manière automatisée, ce qui a permis de constituer un corpus initial.

2.1.2 Transformation des données

Afin de transformer cet échantillon en un jeu de données réaliste et exploitable, ces fichiers bruts sont soumis à un pipeline de transformations reproduisant la diversité des données en mémoire.

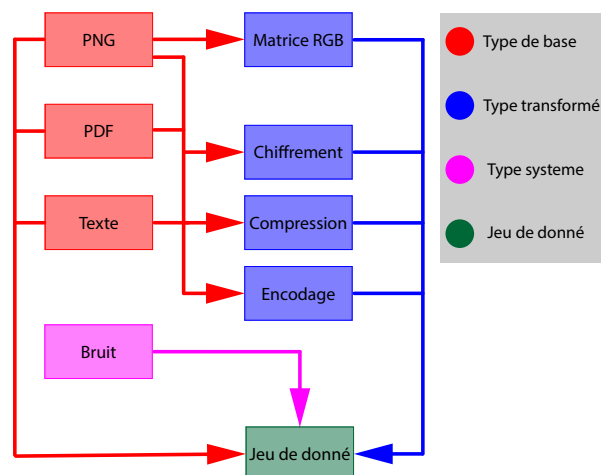


FIGURE 2.1 – Schéma du processus de transformation des données

Nos trois types de données principaux (Texte, PDF, Images) sont soumis à des transformations détaillées dans la [Figure 2.1](#), telles que l'encodage via le Base64, la compression à taux variable via zLib ou Gzip, ou encore le chiffrement via l'algorithme AES-256-CBC. Et la conversion en matrice de pixels RGB pour les images. On crée aussi du bruit aléatoire. Toutes ces transformations sont appliquées avec une seed afin de garantir la reproductibilité du processus tant que les données sources restent inchangées.

Avec toutes ces transformations, nous obtenons une grande variété de sous-classes de données, que nous étiquetons en fonction de leur nature, chiffré ou non chiffré. Cette diversité est essentielle pour entraîner un modèle capable de généraliser à des données réelles, qui peuvent être fragmentées, obfusquées ou structurées de manière inattendue.

Nos deux classes sont donc :

- Non Chiffré
 - Texte brut
 - PDF
 - Image brute, en PNG
 - Image convertie en matrice de pixels RGB
 - Données compressées via algorithme
 - Données encodées via Base64
 - Données bruitées
- Chiffré
 - Les données chiffrées via algorithme

2.1.3 Création du segment RAM synthétique

Tout le processus de création du "dump" est reproductible grâce à l'utilisation d'une seed, garantissant que les mêmes données sources génèrent le même dump synthétique à chaque exécution du pipeline. La répartition des différentes classes est équilibrée, avec 50% de données chiffrées et 50% de données non chiffrées, afin d'éviter tout biais lors de l'entraînement du modèle. Et chaque type des données non chiffrées est également représenté de manière équilibrée pour garantir une diversité suffisante au sein de cette classe, illustré dans la [Figure 2.2](#).

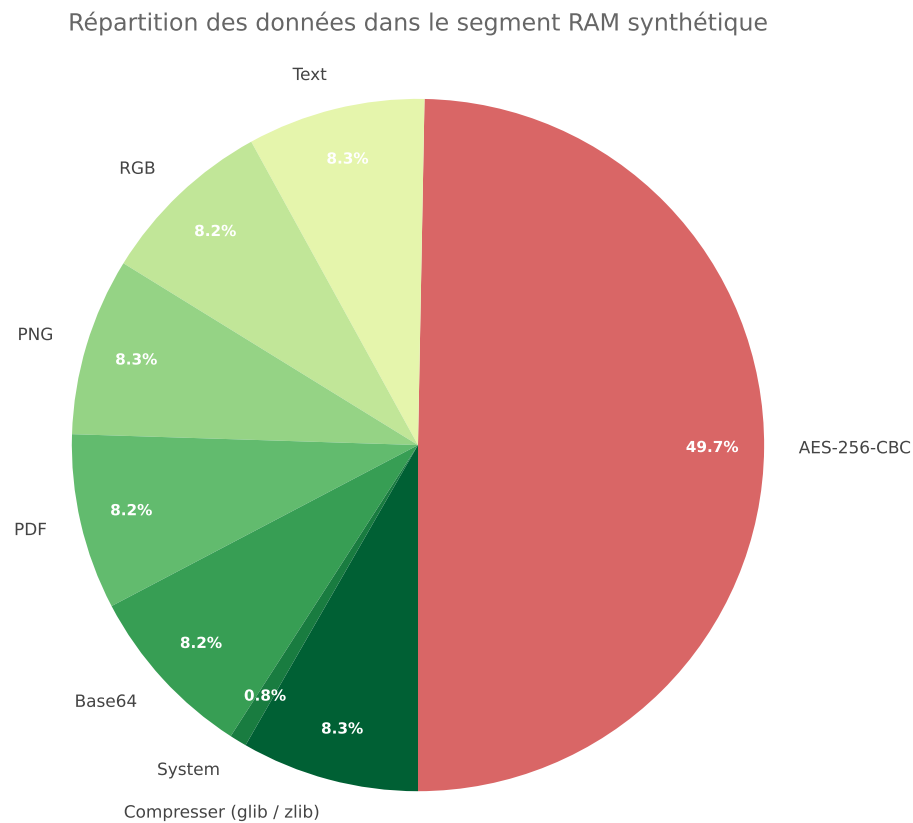


FIGURE 2.2 – Répartition des données dans le dump mémoire synthétique

Une fois transformées, les données sont injectées dans un espace mémoire contigu, le dump synthétique. Chaque fichier est découpé en segments de taille variable, entre 1024 et la taille totale du fichier, afin de simuler la fragmentation naturelle des données en RAM. Les segments sont ensuite mélangés de manière aléatoire pour éviter toute corrélation spatiale entre les différentes classes de données, ce qui rend la tâche de classification plus réaliste et complexe pour le modèle.

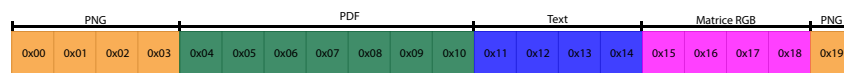


FIGURE 2.3 – Illustration d'un dump RAM synthétique

2.1.4 Construction des différents "dumps"

Avec ce pipeline de génération de données, nous avons construit deux dumps synthétiques de 1,2 Go et 500 Mo, dédiés respectivement à l'entraînement et au test du modèle. Ces dumps sont équilibrés, avec la répartition de la [Figure 2.2](#). Pour le dump d'entraîne-

ment, 20% des données sont réservées à la validation, afin de suivre l'évolution de la performance du modèle et d'éviter le surapprentissage. Le dump de test, quant à lui, est utilisé pour évaluer la capacité du modèle à généraliser à des données qu'il n'a jamais vues auparavant.

2.2 LE DÉCOUPAGE EN CHUNKS

Pour permettre au modèle de traiter efficacement les données, le dump est découpé en segments contigus de taille fixe, appelés "chunks". Chaque chunk contient 1024 octets, ce qui correspond à la taille maximale que le modèle peut traiter en une seule passe. Ce découpage permet de structurer les données de manière à ce que le modèle puisse apprendre à identifier des motifs locaux au sein de chaque segment. Cependant, ce découpage en chunks introduit une discontinuité artificielle dans les données, car les frontières entre les chunks peuvent couper des structures de données ou des motifs importants. Pour compenser cette limitation, nous avons implémenté une stratégie de "overlapping" (recouvrement) lors de l'inférence, qui sera détaillée dans la [Section 2.6.2](#).

2.3 MODÈLE DE CLASSIFICATION

2.3.1 Introduction au principe du *Transformer*

L'architecture *Transformer* représente un changement de paradigme majeur dans le domaine de l'apprentissage profond (Deep Learning). Introduit par *Google* en 2017 à travers l'article «*Attention is All you Need*», ce modèle s'est imposé comme le standard pour le traitement de données séquentielles, notamment dans le traitement automatique des langues (NLP).

Contrairement aux architectures récurrentes conventionnelles (RNN et LSTM), qui opèrent par une analyse chronologique et itérative de l'information, le *Transformer* repose sur un principe de parallélisation massive des flux de données. Ce qui permet de traiter des séquences entières simultanément, sans dépendre de l'ordre d'entrée. Le cœur de cette architecture réside dans le mécanisme d'attention, qui permet au modèle de pondérer l'importance relative de chaque élément d'une séquence par rapport à tous les autres, facilitant ainsi la capture de dépendances à longue distance.

Architecture Encodeur

Un encodeur traite et modélise l'information d'entrée en convertissant des données brutes en représentations vectorielles de haute dimension. Lorsqu'il est utilisé indépendamment d'un décodeur, ce module est particulièrement efficace pour les tâches de classification ou de compréhension sémantique, où la génération de texte n'est pas requise.

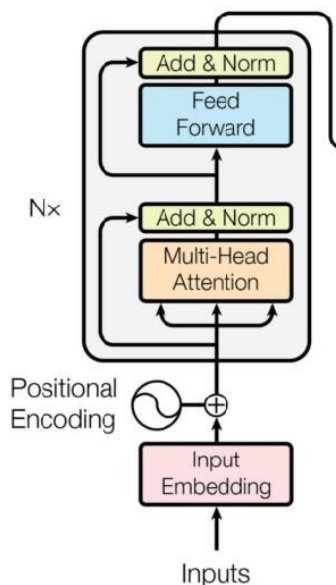


FIGURE 2.4 – Illustration d'un encodeur du papier « *Attention is All you Need* »

Input Embedding

L'*input embedding* transforme les données brutes en représentations vectorielles de dimension d_{model} , rendant l'information traitable mathématiquement par le modèle.

Une fois projetés dans l'espace vectoriel, ces vecteurs sont multipliés par un facteur d'échelle $\sqrt{d_{model}}$ afin de stabiliser les gradients lors de l'entraînement.

Contrairement aux réseaux récurrents (RNN), l'architecture *Transformer* traite les éléments d'une séquence simultanément. Pour compenser cette absence de notion d'ordre intrinsèque, un encodage de position est ajouté à chaque vecteur d'entrée. Cette information permet au modèle de distinguer la position relative de chaque élément dans la séquence.

Cet encodage repose sur des fonctions sinusoïdales de fréquences différentes, définies

par les formules suivantes :

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (2.1)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (2.2)$$

Mécanisme d'attention

Le mécanisme de *self-attention* permet au modèle de pondérer l'importance relative de chaque élément d'une séquence par rapport à tous les autres au sein de la même entrée. En projetant chaque token dans trois espaces distincts (Q, K, V), le modèle calcule une distribution de scores par produit scalaire. Cette opération permet d'extraire les dépendances contextuelles, qu'elles soient locales ou à longue distance, en focalisant le traitement sur les parties les plus informatives de la séquence.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.3)$$

Q = matrice de requêtes

K = matrice de clés

V = matrice de valeurs

d_k = dimension des clés

L'attention multi-têtes, c'est l'application de plusieurs mécanismes de *self-attention* en parallèle, dans le but de spécifier et d'extraire plusieurs métriques (types, syntaxe ...) afin de ne pas écarter de sens exploitable. Leurs résultats sont donc fusionnés pour extraire un unique vecteur portant toutes les informations sur le sens.

Sortie du modèle

A la sortie du modèle on trouve des logits (score une valeur entre $-\infty$ et $+\infty$), nous y appliquons une fonction sigmoïde pour *Transformer* entre 0 et 1, entre les deux classes.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.4)$$

2.4 BytesTransformerClassifier

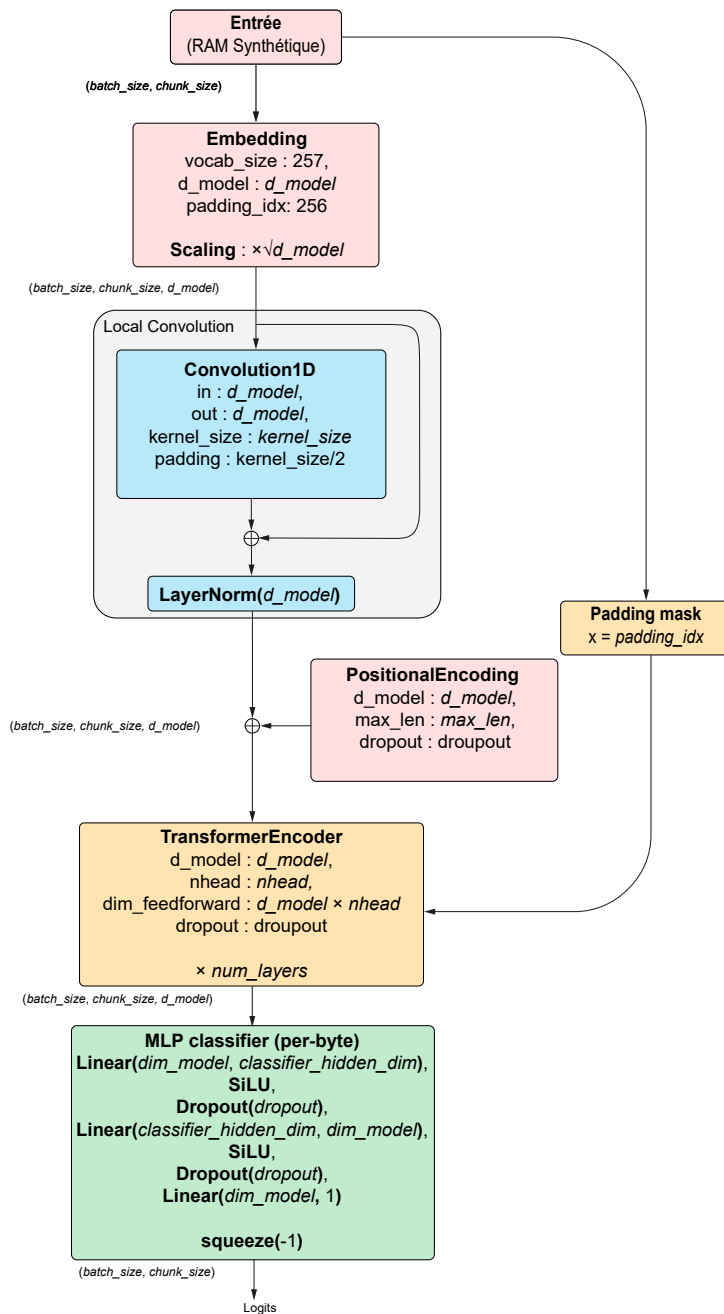


FIGURE 2.5 – Architecture du réseau : BytesTransformerClassifier

Paramètre	Valeur
chunk_size	1024
d_model	256
kernel_size	3
num_heads	8
num_layers	4
dropout	0.2
classifier_hidden_dim	256

TABLE 2.1 – Paramètres du classifieur RAID

Nous avons opté pour une architecture hybride combinant une couche de Convolution unidimensionnelle (*Local Convolution* dans la [Figure 2.5](#)) en amont du bloc d’encodeur. Cette configuration permet d’extraire des motifs locaux au sein des séquences d’octets, tout en bénéficiant de la capacité du *Transformer* à modéliser les dépendances à longue distance.

2.5 ENTRAÎNEMENT ET VALIDATION

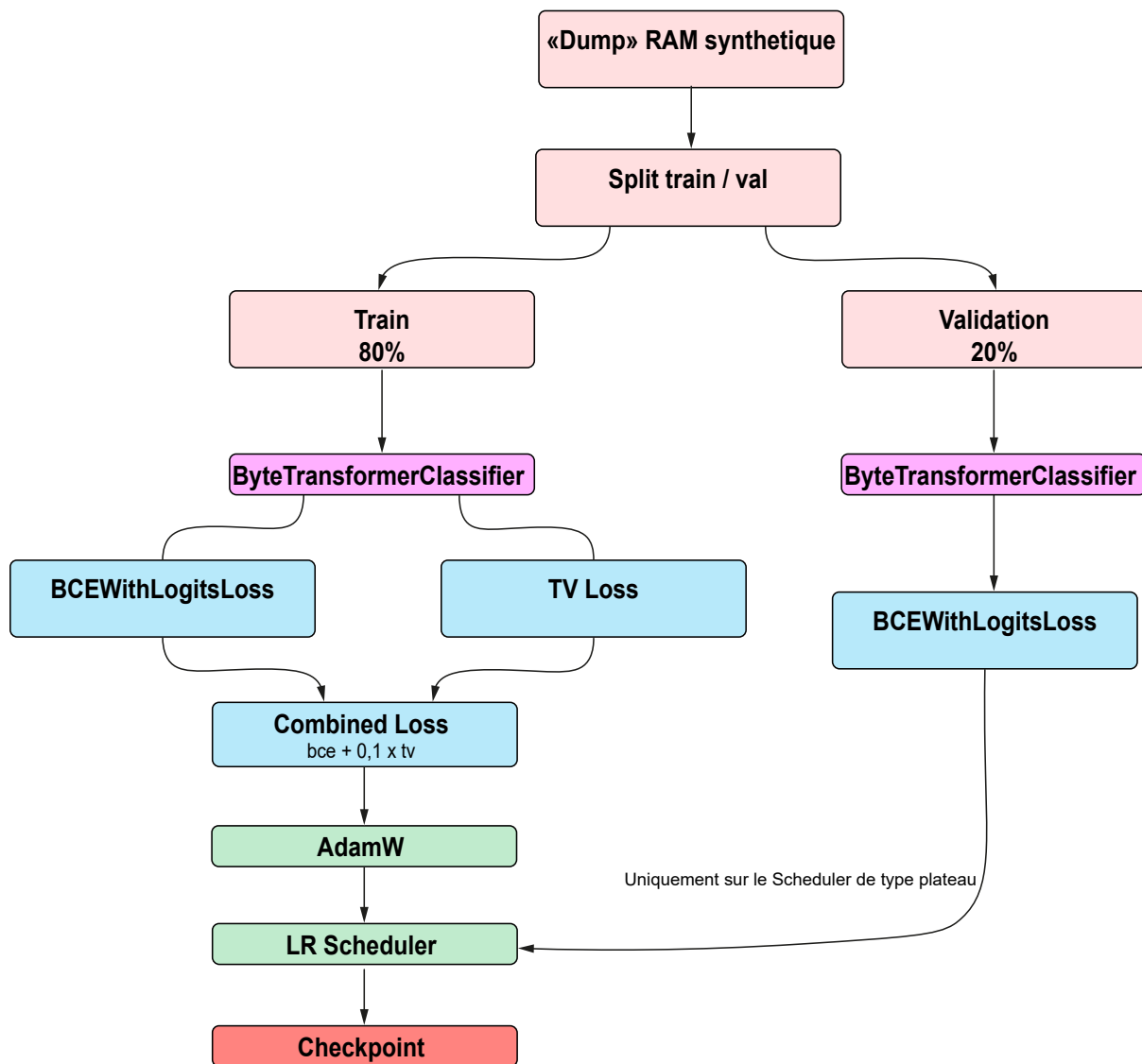


FIGURE 2.6 – Processus d’entraînement du modèle

Le "dump" est divisé en deux parties avec un `random_split`. 80% des données sont utilisées pour l’entraînement, tandis que les 20% restants sont réservés à la validation. Le split est reproductible grâce à l’utilisation d’une seed fixe.

Ensuite, nous calculons la loss principale avec `BCEWithLogitsLoss`, qui combine une fonction de perte binaire avec une activation sigmoïde intégrée. Nous avons aussi ajouté une TV loss (Total Variation Loss) pour encourager la cohérence spatiale des prédictions, en pénalisant les changements de classe abrupts entre octets adjacents. La loss totale est une

combinaison pondérée de ces deux composantes, avec un coefficient λ , ici de 0.1, ajusté pour équilibrer leur influence.

Nous utilisons l'optimiseur AdamW, qui est une variante de l'Adam classique, intégrant une régularisation par poids (weight decay) pour améliorer la généralisation du modèle. Le taux d'apprentissage initial est fixé à $1e-4$. Nous utilisons aussi un scheduler de type CosineAnnealingLR pour ajuster dynamiquement le taux d'apprentissage au cours de l'entraînement, permettant ainsi une convergence plus efficace. Le scheduler plateau était moins efficace. Le warmup linéaire peut être chaîné avant le scheduler principal (sauf avec Plateau) via SequentialLR.

A la fin de chaque epoch, on save le modele dans un chekpoint

Pour choisir le meilleur modèle, nous suivons la métrique d'accuracy et le loss sur l'ensemble de validation. Le modèle avec la meilleure accuracy et la meilleure loss est sélectionné pour l'inférence finale sur le dump de test.

2.6 INFERENCE

2.6.1 Processus d'inférence

Avec le meilleur modèle sélectionné, nous procédons à l'inférence sur le dump de test. Lors de l'inférence, le dump de RAM est traité par segments de 1024 octets, conformément à la taille d'entrée du modèle. Chaque segment est passé à travers le classifieur, qui génère une prédiction binaire pour chaque octet, indiquant s'il est chiffré ou non.

2.6.2 Principe de *l'overlapping* (recouvrement)

Le modèle traite le dump mémoire par fenêtres glissantes de N octets. Au lieu de découper la mémoire en blocs contigus, nous utilisons un pas de glissement (stride) inférieur à la taille de la fenêtre. Par conséquent, chaque octet de la RAM est analysé plusieurs fois par le Transformer, au sein de contextes différents.

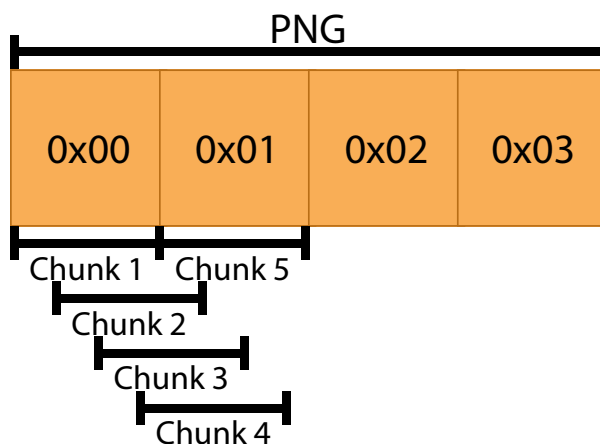


FIGURE 2.7 – Illustration du principe de l’overlapping, ici avec un stride de $\frac{1}{4}$ de chunk

Après avoir transformé les logits en probabilités via une fonction sigmoïde, les prédictions locales sont agrégées. Cette étape repose sur une pondération de chaque résultat par le score de confiance associé.

$$\text{Confiance}_x = 2 \times |p_x - 0.5| \quad (2.5)$$

$$P_Global_x = \frac{\sum_{i=1}^n \text{Confiance}_i \times p_i}{\sum_{i=1}^n \text{Confiance}_i} \quad (2.6)$$

$$\text{avec } n = \frac{\text{chunk_size}}{\text{stride}}$$

2.7 OPTIMISATION

Afin d’aller plus loin sur le projet, nous avons eu besoin de monter significativement nos volumes de données synthétiques pour le train ainsi que le nombre d’époques. Nous avons donc dû trouver des optimisations afin d’exploiter le plus possible notre matériel.



FIGURE 2.8 – Goulot d’étranglement entre le CPU et le GPU avant optimisation, visualisé avec *NVIDIA Nsight Systems*

Premièrement nous avons identifié un goulot d’étranglement entre le CPU et le GPU, comme nous pouvons le voir sur la Figure 2.8. Afin de résoudre ce problème nous avons configuré des options de parallélisation avancées sur les Dataloaders notamment via les `persistent_workers`, un `prefetch_factor` élevé et une refonte du code lié au `__getitem__`. Comme nous pouvons le voir sur la Figure 2.9, cela a fonctionné comme espéré.

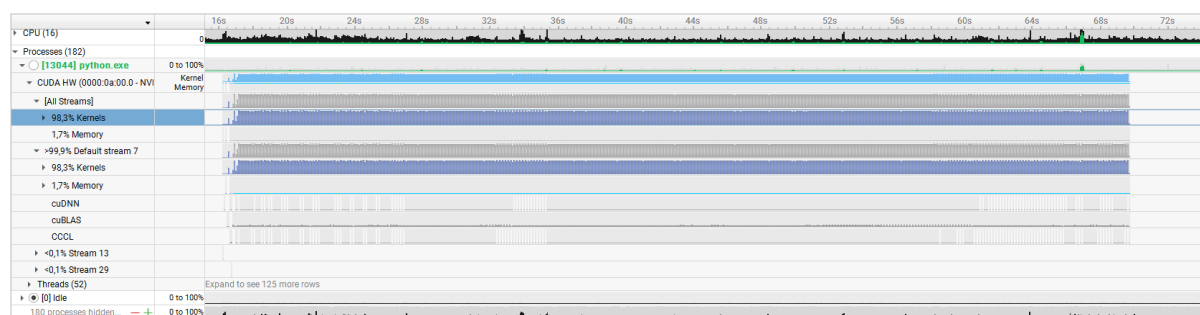


FIGURE 2.9 – Parallélisation efficace du pipeline de données après optimisation, visualisé avec *NVIDIA Nsight Systems*

Dans un second temps, nous avons maximisé la vitesse d’entraînement. Nous avons adopté la précision mixte, *bfloat16*, permettant de réduire la consommation de VRAM par deux et d’accélérer le temps d’entraînement par deux. De plus, nous avons activé `torch.compile` avec le backend Inductor en mode `max-autotune`. Contrairement à une approche standard, ce mode réalise un benchmark exhaustif sur notre architecture, une *RTX 5080* : le compilateur génère et exécute physiquement des variantes de noyaux Triton pour chaque segment du graphe de calcul. Cette recherche identifie la stratégie de fusion de noyaux offrant la latence la plus faible. Bien que cette phase d’autotuning ajoute environ 400 secondes au lancement, elle génère un gain de performance de 27,45% par époque, elle est donc intéressante sur l’entraînement mais également sur des inférences qui seraient longues.

Enfin, la Total Variation Loss a été réimplémentée sous forme de convolution 1D. Cette approche exploite pleinement la parallélisation massive des cœurs CUDA, rendant ce calcul quasi instantané par rapport à une approche itérative classique sur CPU.

2.8 OUTILS D'ANALYSE

Pour analyser les résultats nous avons développé un visualiseur de données, qui permet de superposer les classifications du modèle à la vérité. Il s'agit d'une interface graphique web développée en *React* avec un `<canvas>`.

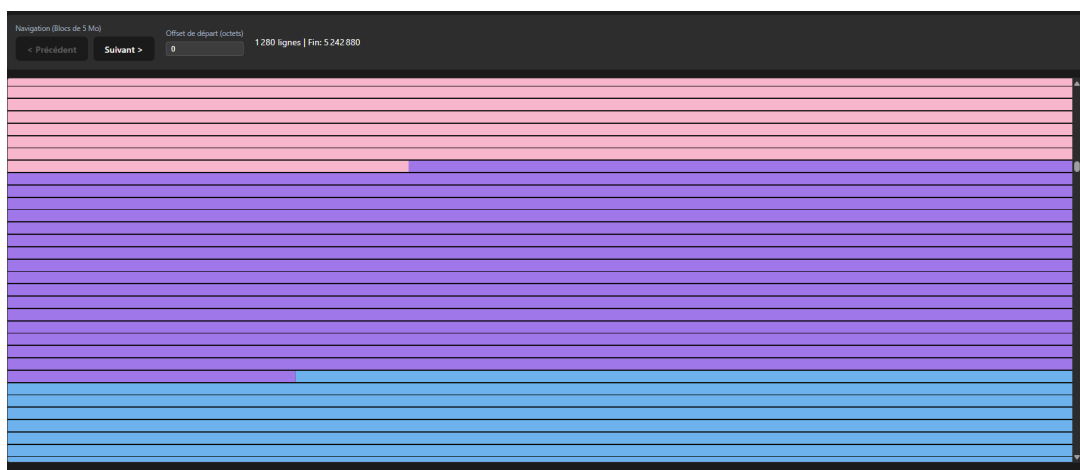


FIGURE 2.10 – Interface de visualisation des données

Le visualiseur cartographie visuellement la mémoire en attribuant une couleur spécifique à chaque type de données (PDF, image, texte, etc.), comme dans la [Figure 2.10](#). En superposant les classifications du Transformer à la vérité terrain, l'outil met en surbrillance rouge les erreurs, la [Figure 2.11](#), offrant ainsi une capacité d'analyse granulaire à l'octet près.

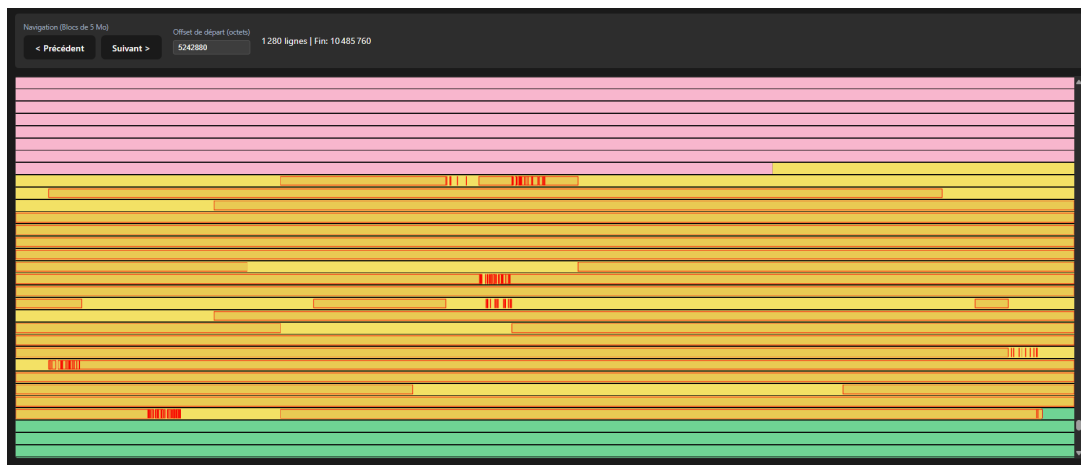


FIGURE 2.11 – Interface de visualisation des erreurs de classification du modèle

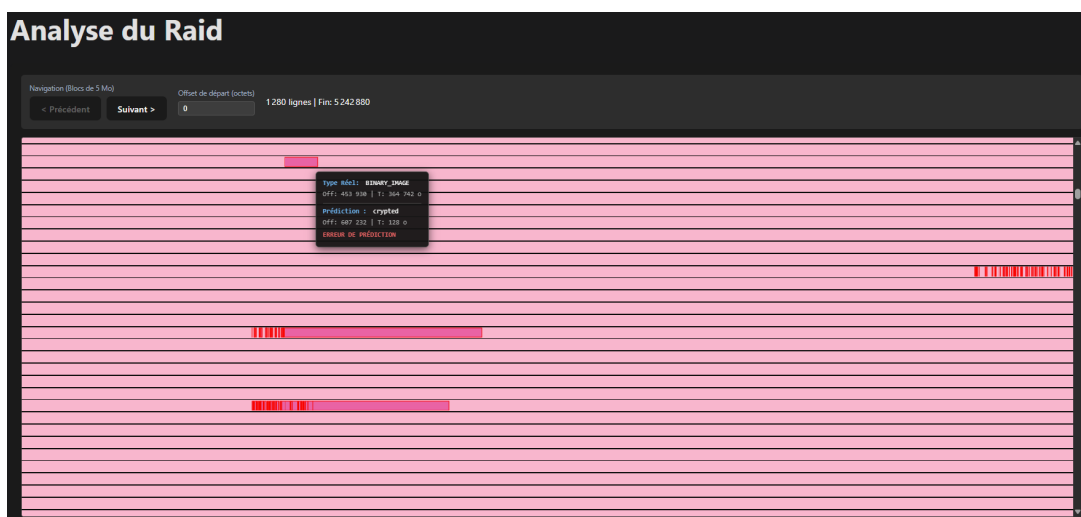


FIGURE 2.12 – Interface de visualisation des détails d'une erreur de classification

Sur ces segments, où les erreurs sont surlignées en rouge, il est possible d'avoir un détail sur l'erreur, la [Figure 2.12](#). De plus, en cliquant sur un segment, il est possible d'accéder directement au détail de ce bloc, c'est-à-dire la donnée brute sous forme hexadécimale, montré dans la [Figure 2.13](#). Cela permet ainsi d'identifier éventuellement des suites vite dans une donnée par exemple.

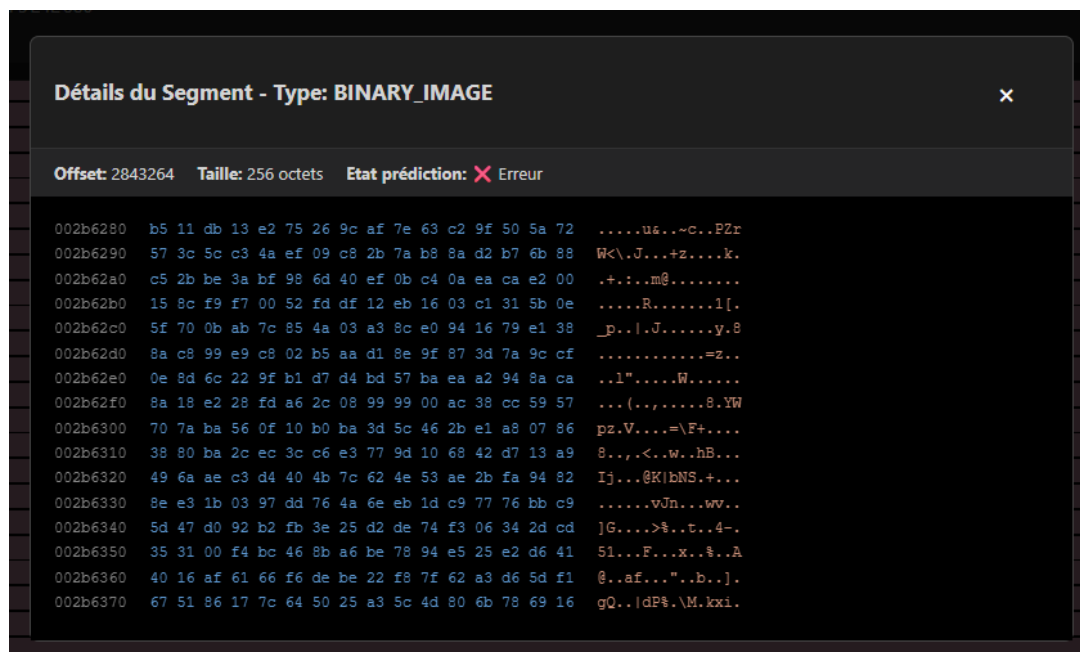


FIGURE 2.13 – Interface de visualisation du détail d'une erreur de classification

C'est l'exploitation de cet outil qui nous a permis d'observer visuellement que les erreurs se concentraient principalement aux frontières des segments, lors des transitions entre deux types de données. Cette découverte a directement motivé l'implémentation du mécanisme de vote pondéré détaillé précédemment.

3 Résultats

3.1 LE VOTE PONDÉRÉ

Accuracy		Diminution de l'erreur
Sans vote pondéré	Avec vote pondéré	
87,59%	88,79%	9.67%

TABLE 3.1 – Performances du vote pondéré

Comme on peut le lire que le **Tableau 3.1**, le vote pondéré a permis d'améliorer l'accuracy de 1,2 points, soit une réduction de l'erreur de 9,67%. Cette amélioration significative souligne l'efficacité de cette stratégie pour stabiliser les prédictions du modèle et réduire les artefacts de classification.

3.2 COMPARAISON DES CONFIGURATIONS DU MODÈLE

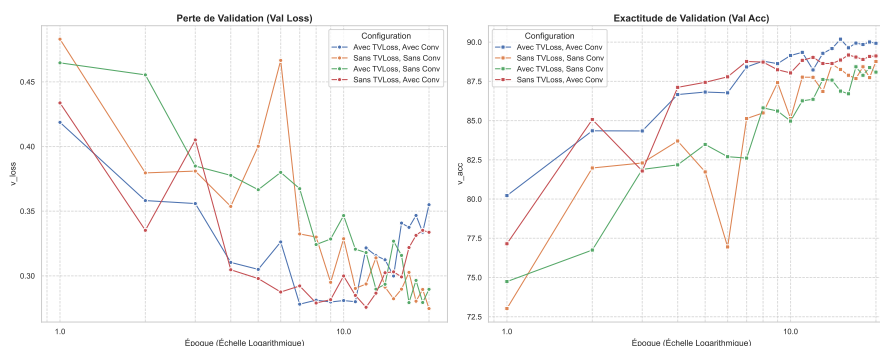


FIGURE 3.1 – Courbes d'apprentissage du modèle (axes des abscisses en échelle logarithmique)

Les resultats ci-dessus, la **Figure 3.1**, sont obtenu sur des petits jeux de données, 50 Mo, afin de montrer une tendance de l'utilisation ou non du *TV Loss* ou de la convalution. La configuration avec *TVLoss* et Convolution (courbe bleu foncé) se distingue comme étant la plus performante sur l'ensemble de la période d'entraînement. En observant l'évolution de l'exactitude de validation, cette configuration atteint les niveaux les plus élevés, culminant au-delà de 90% d'exactitude vers la fin des 20 époques. Parallèlement, elle démontre une stabilité remarquable dans la réduction de la perte de validation, se maintenant aux alentours de 0.30 après une phase de stabilisation initiale. À l'inverse, les configurations dépourvues

de convolution présentent une plus grande instabilité, avec des fluctuations marquées de la perte et une progression plus lente de l'exactitude, soulignant l'apport crucial de l'architecture convolutive pour la convergence du modèle.

Ici on aurait donc choisi le modèle, avec *TVLoss* et Convolution, à l'époque 15, qui présente un bon compromis entre l'exactitude et la perte.

3.3 PERFORMANCE GLOBALES

3.3.1 Matrices de confusion

Les performances du modèle ont été analysées plus finement avec des matrices de confusion, à la fois au niveau global (chiffré vs non chiffré) et au niveau des sous-classes (texte, PDF, image, données compressées, données chiffrées). Ces matrices permettent d'identifier les types de données qui sont les plus souvent confondus par le modèle.

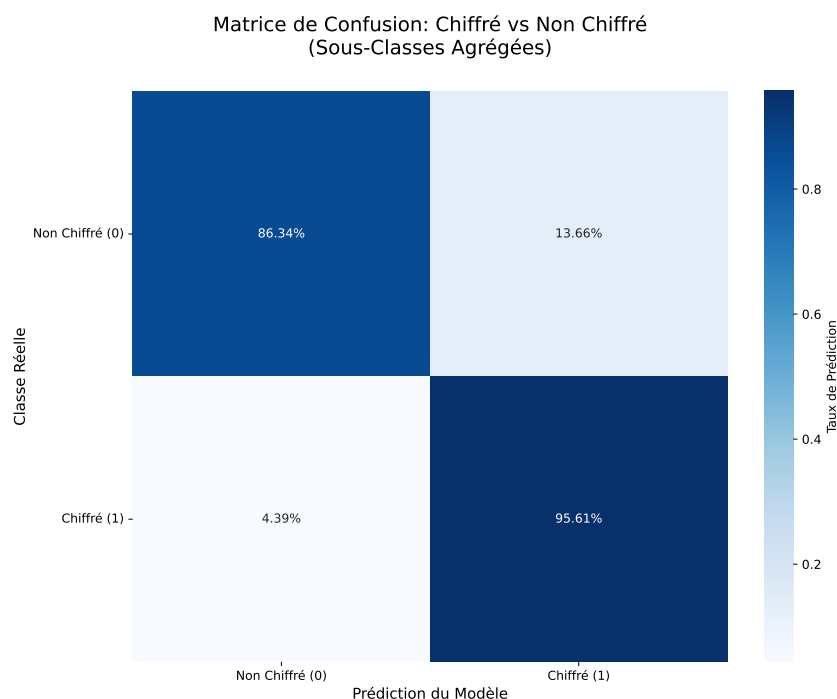


FIGURE 3.2 – Matrice de confusion du modèle

Comme le montre la [Figure 3.2](#), le modèle atteint une précision de 95,61% sur la détection de contenus chiffrés et de 86,34% pour les contenus non chiffrés. Cela indique que le modèle est plus performant pour identifier les données chiffrées, avec un faible taux de faux négatifs, de 4,39%.

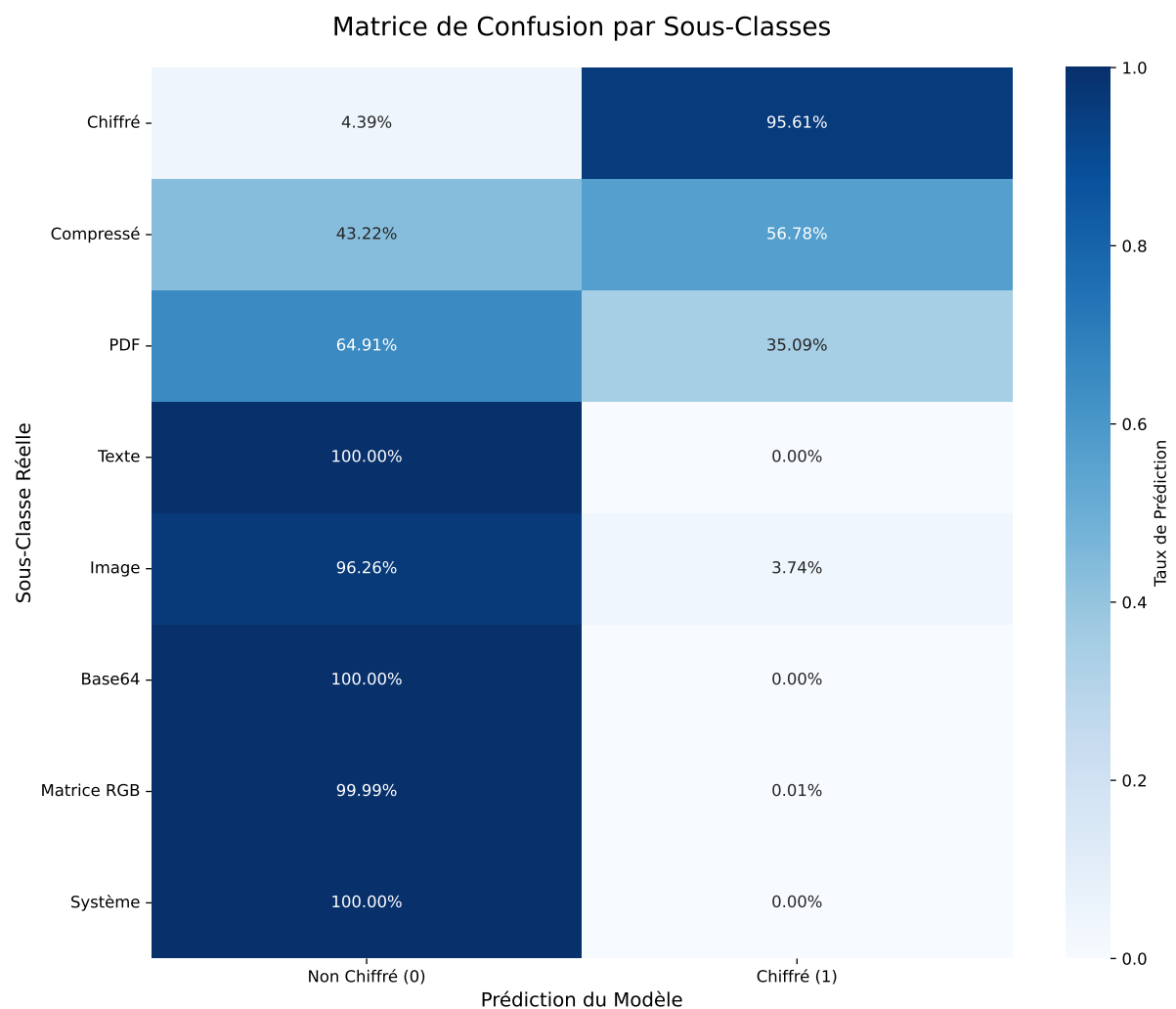


FIGURE 3.3 – Matrice de confusion du modèle, détaillé par type de données

Pour comprendre ces erreurs, nous avons décomposés les résultats sur chaque type de données, la **Figure 3.3**. On observe que les données compressées sont confondues avec des données chiffrées. En revanche, les données textuelles, les PDF et les images sont très bien différenciés du chiffrement, avec des taux de précision respectifs de proche de 100%.

3.3.2 Métriques d'évaluation

Métrique finale	Score
Accuracy	88,79%
Precision	$\approx 86\%$
Recall	85,21%
F1-Score	85,60%

TABLE 3.2 – Performances globales du classifieur RAID

[NEED TO CHANGE] : Ajouter un explicatif sur les métriques

4 Discussion

[NEED TO CHANGE] : Pas touché

4.1 PERFORMANCE DU VOTE PONDÉRÉ

L'un des défis majeurs de l'analyse de dumps mémoire par un *Transformer* réside dans la volatilité des prédictions locales. Un segment de données peut présenter des caractéristiques ambiguës (par exemple, un en-tête de fichier compressé ressemblant à du texte structuré) qui induisent le modèle en erreur s'il est analysé de manière isolée.

Pour pallier cette limite, nous avons implémenté un mécanisme de vote pondéré. Cette approche permet de stabiliser les prédictions et d'augmenter la fiabilité de la classification des zones de mémoire.

En confrontant plusieurs prédictions pour un même octet via des fenêtres glissantes, nous avons observé une réduction des "artefacts de prédiction" (changements de classe abrupts et incohérents sur quelques octets). Cette approche agit comme un filtre passe-bas, éliminant le bruit haute fréquence dans les résultats pour ne conserver que les zones où le modèle maintient une confiance élevée et constante.

Il est important de noter que cette robustesse a un coût. L'utilisation d'un *stride* (pas) réduit pour multiplier les votes multiplie proportionnellement le temps d'inférence. Dans des grands volumes de données, ce compromis entre le temps d'analyse et la précision de la segmentation est un paramètre critique. Nos résultats suggèrent que le gain en fiabilité justifie cette latence, tout en ouvrant la porte à des optimisations futures sur le parallélisme de l'agrégation des votes.

4.2 CHIFFREMENT ET COMPRESSION : L'ENTROPIE TROMPE LE MODÈLE

L'analyse des erreurs de classification révèle une confusion récurrente du modèle entre les segments de données chiffrés et compressés. Cette difficulté s'explique par la nature mathématique même des algorithmes utilisés dans le projet, tels que l'AES-256-CBC pour le chiffrement et zLib ou Gzip pour la compression.

4.2.1 La convergence statistique des hautes entropies

Le modèle *Transformer* repose sur sa capacité à identifier des structures et des dépendances au sein des séquences d'octets. Or, l'objectif d'un algorithme de compression efficace est d'éliminer toute redondance, ce qui conduit à une distribution des octets proche d'un bruit blanc aléatoire, simulant ainsi une entropie maximale. Le chiffrement et la compression sont deux processus produisant des flux binaires dépourvus de motifs répétitifs exploitables par les mécanismes d'attention du modèle.

4.2.2 Impact sur la précision de la segmentation

Dans les résultats d'évaluation, cette confusion se traduit par des faux positifs où des flux compressés sont étiquetés comme ENCRYPTED. Les résultats montrent que même avec une validation par vote pondéré, le score de confiance reste souvent élevé pour ces deux catégories, car le modèle "est certain" de ne pas faire face à du texte ou des images structurées, mais ne possède pas de descripteurs sémantiques suffisants pour distinguer un flux compressé d'un flux chiffré.

4.2.3 Limites de l'approche sémantique pure

Cette problématique souligne une limite intrinsèque de l'apprentissage profond appliqué aux flux binaires bruts : sans la détection de signatures spécifiques que le projet cherche justement à éviter pour gagner en généricité, la distinction entre haute entropie "utile" (compression) et "intentionnelle" (chiffrement) demeure un défi ouvert. Pour les travaux futurs, l'intégration de métriques statistiques complémentaires, telles que des tests de distribution de fréquences bien plus fins au sein des couches du classifieur, pourrait offrir une piste pour séparer ces deux types de données.

4.2.4 Sémantique binaire vs Identification rigide

Les méthodes classiques échouent lorsque les données sont fragmentées ou dépourvues de leurs en-têtes originaux, une situation fréquente au sein d'un dump RAM. Là où un outil statique verrait une suite d'octets incohérente, le modèle parvient à extraire une "signature statistique" du contenu. En apprenant la sémantique profonde des différents types de données (la structure d'un texte, la distribution d'une image compressée), le *Transformer* identifie la nature de l'information même en l'absence de métadonnées explicites.

4.2.5 Résilience face à l'obfuscation

Une limite majeure des signatures statiques réside dans leur vulnérabilité aux techniques d'obfuscation simples. Un changement mineur dans les premiers octets d'un fichier suffit à tromper un scanner traditionnel. À l'inverse, l'utilisation du mécanisme d'attention du *Transformer* permet au modèle de pondérer l'importance de chaque octet dans son contexte global. Cette capacité rend le système beaucoup plus résilient : la classification ne repose pas sur un point de passage unique, mais sur une compréhension diffuse du bloc de données de 1024 octets.

4.2.6 Le coût de l'intelligence

Il est toutefois nécessaire de nuancer cette supériorité par le coût computationnel associé. Si les signatures statiques sont quasi instantanées et légères, l'inférence d'un modèle *Transformer* couplée à une stratégie de vote pondéré, exige des ressources matérielles significatives (GPU). Cette discussion souligne un arbitrage nécessaire : les méthodes classiques restent d'une utilité majeure en l'absence de matériel performant ou lors d'analyses où l'on cherche la rapidité brute.

4.3 LIMITES

4.3.1 La généralisation face aux données réelles

Bien que les résultats soient prometteurs, il est important de souligner que le modèle a été entraîné et évalué sur un jeu de données synthétique. Bien que ce dernier s'efforce de reproduire la diversité d'un dump mémoire, il existe inévitablement des écarts avec la complexité d'une mémoire vive réelle, notamment au niveau de la fragmentation dynamique et des structures de données propriétaire liées aux OS.

On peut aussi considérer que dans la RAM, la présence de formats propriétaires non inclus dans le dataset pourraient faire l'objet d'erreurs de classification si leur format paraît trop éloigné de la signature statistique des données présentes dans le corpus d'entraînement. La non exhaustivité des types présents dans notre dataset peut donc influencer négativement les performances de la classification du modèle.

4.3.2 Le choix de l'architecture *Transformer*

Le passage d'une analyse par signatures à une analyse par apprentissage profond a nécessité le choix d'une architecture capable de saisir la complexité sémantique des flux binaires. Si les réseaux de neurones récurrents (RNN ou LSTM) ont longtemps été la norme pour les données séquentielles, le choix d'un *Transformer* pour le projet R.A.I.D. répond à des problématiques spécifiques à l'analyse de la mémoire vive.

***Multi-Head Attention* vs Dépendances séquentielles**

La principale limite des RNN réside dans leur nature séquentielle : ils traitent les octets l'un après l'autre, ce qui rend difficile la capture de relations à longue distance au sein d'un bloc de données. À l'inverse, le mécanisme d'attention multi-têtes permet au modèle d'analyser simultanément toutes les relations entre les octets d'une fenêtre de 1024 octets. Cette capacité est cruciale : un en-tête situé au début d'un segment peut avoir une influence sémantique directe sur des données situées bien plus loin. Là où un RNN aurait pu "oublier" le contexte initial, le Transformer maintient une vision globale et pondère l'importance de chaque zone du dump avec une précision accrue.

L'importance de l'encodage positionnel

Contrairement aux images ou au texte naturel, la structure d'un dump mémoire est strictement liée à l'adresse physique des données. L'absence de structure inhérente aux *Transformers* (qui sont, par nature, invariants par permutation) a été compensée par l'implémentation d'un Positional Encoding. Cette composante n'est pas qu'un détail technique : elle permet au modèle de conserver la notion d'ordre et de distance entre les octets. Dans notre contexte, savoir qu'un octet spécifique se situe à une position relative précise par rapport à un autre est souvent le seul moyen de distinguer un motif structuré d'un flux aléatoire.

Parallélisation et passage à l'échelle

Enfin, un argument pragmatique a dicté ce choix : la performance. L'architecture *Transformer* permet une parallélisation totale lors de l'entraînement et de l'inférence, contrairement aux RNN qui sont limités par leur récursivité. Étant donné la volumétrie des dumps RAM modernes (souvent supérieurs à 16 Go), l'utilisation des cœurs de calcul des GPU via une architecture *Transformer* est la seule voie viable pour obtenir des temps d'analyse convenables.

4.4 PERSPECTIVES

4.4.1 Passage en apprentissage non supervisé

Actuellement, le système repose sur un entraînement supervisé à l'aide d'un générateur de datasets qui définit arbitrairement les classes de données. Cependant, la réalité d'un dump RAM système est bien plus complexe et hétérogène que les échantillons produits synthétiquement. Un passage à l'apprentissage non supervisé permettrait au modèle d'apprendre directement les structures statistiques intrinsèques de la mémoire vive sans nécessiter d'étiquetage préalable, facilitant ainsi la détection d'artefacts encore inconnus.

L'architecture *Transformer* de R.A.I.D. est particulièrement adaptée à cette transition via des techniques de pré-entraînement similaires à celles utilisées dans les modèles de langage (type BERT). En masquant certains octets au sein d'une séquence et en demandant au modèle de les prédire (architecture decoder), le système développerait une compréhension profonde de la "syntaxe" binaire des différents types de données. Dans ce cadre, une erreur de prédiction élevée deviendrait un indicateur natif d'une zone à haute entropie (chiffrement), sans que le modèle ait besoin d'avoir "vu" des données chiffrées durant son entraînement.

L'objectif final de cette perspective est de permettre au modèle de s'adapter dynamiquement à n'importe quel système d'exploitation ou architecture matérielle. En passant au non supervisé, R.A.I.D. ne chercherait plus à valider des catégories pré-définies, mais à segmenter la mémoire en zones de cohérence. Cette approche permettrait d'isoler avec une plus grande précision les zones de données volatiles, posant ainsi les bases d'une assistance automatisée capable de s'auto-ajuster à la spécificité de chaque dump RAM analysé, bien que cela reste hypothétique.

4.4.2 Le défi de la fragmentation et de la reconstruction

Si le projet R.A.I.D. a validé la capacité des *Transformers* à classifier des segments isolés, une étape suivante pourrait consister en la gestion de la fragmentation et la reconstruction logicielle des fichiers. Un dump RAM n'est pas une suite linéaire de fichiers, mais un agglomérat de blocs mémoires souvent éparpillés.

De la segmentation locale à la vision globale

Le modèle actuel travaille sur des fenêtres de 1024octets (*chunk_size*), ce qui lui permet d'identifier avec précision la nature d'un fragment (image, PDF, texte). Le défi est de déter-

miner si deux segments identifiés comme IMAGE et situés à des adresses distantes appartiennent au même objet binaire. L'utilisation du mécanisme d'attention pourrait être étendue pour comparer les "signatures sémantiques" de différents blocs afin de détecter des continuités logiques malgré la fragmentation physique.

R.A.I.D. comme moteur de "File Carving" intelligent

Les outils de *file carving* traditionnels (Scalpel, Foremost ..) reposent sur la détection de structures rigides (en-têtes et pieds de fichiers). R.A.I.D. offre une perspective de "*carving* sémantique" via :

- L'isolation des zones d'intérêt au milieu du bruit ou des données chiffrées, le modèle réduira drastiquement l'espace de recherche pour les algorithmes de reconstruction.
- La reconstruction sans en-tête, car le modèle a prouvé sa capacité à reconnaître des structures (comme des pixels bruts via DECODED) même lorsque les métadonnées du fichier original ont disparu de la RAM. Cela permettrait de récupérer des documents partiels là où les méthodes classiques échouent.

Vers une assistance à la reconstruction automatique

À terme, l'intégration de R.A.I.D. dans une chaîne de traitement automatisée permettrait de proposer une prévisualisation des fichiers fragmentés. En couplant la classification par octets avec des techniques de réassemblage basées sur la probabilité de transition entre les types de données, le projet pose les bases d'un outil capable de reconstituer des preuves volatiles (fragments de conversations, images de sessions) qui sont aujourd'hui souvent considérées comme perdues car trop déstructurées.

5 Conclusion

[NEED TO CHANGE] : Pas touché, faut une problématique que tout le monde soit d'accord

Le projet R.A.I.D. (RAM Artificial Inspection Dump) a permis de démontrer l'efficacité des architectures *Transformer* pour l'analyse automatisée de la mémoire vive, un domaine où la volumétrie et la fragmentation des données rendent les méthodes traditionnelles de moins en moins opérantes. En nous affranchissant des signatures statiques au profit d'une approche basée sur la compréhension statistique des flux binaires, nous avons développé un outil capable de segmenter un dump mémoire avec une précision de 88,79%.

L'implémentation du modèle `BytesTransformerClassifier`, couplée à une couche de convolution locale et à un mécanisme de *self-attention*, a prouvé sa capacité à identifier la nature profonde des données (texte, images, documents) même en l'absence de métadonnées ou d'en-têtes explicites. L'introduction du pipeline de vote pondéré a constitué une étape clé de notre méthodologie, permettant de stabiliser les prédictions et de réduire significativement le bruit de classification inhérent à l'analyse par fenêtres glissantes.

Toutefois, ce projet a également mis en lumière des défis persistants, notamment la convergence statistique entre les données chiffrées et compressées. Cette limite souligne que si l'apprentissage profond excelle dans la reconnaissance de structures ordonnées, la distinction entre différentes formes de haute entropie nécessite encore l'intégration de métriques mathématiques plus fines ou de contextes d'analyse élargis.

En conclusion, R.A.I.D. pose les jalons d'une nouvelle génération d'outils de forensique assistée par intelligence artificielle. Les perspectives ouvertes par ce travail qu'il s'agisse du passage à l'apprentissage non supervisé pour s'adapter dynamiquement à de nouveaux systèmes, ou du développement d'un moteur de *file carving* sémantique suggèrent que l'IA peut transformer radicalement la manière dont les enquêteurs extraient des preuves volatiles d'un système. Ce projet démontre que, face à la complexité croissante des environnements numériques, la capacité des modèles de langage à "lire" le binaire brut constitue un atout majeur pour l'informatique légale de demain.

Bibliographie

- GOUGEON, Thomas. « Analyse forensique de la mémoire des cartes à puce ». 2017, theses.hal.science/tel-01661305/document.
- VASWANI, Ashish, et al. « Attention is All you Need ». *Advances in Neural Information Processing Systems (NIPS)*. 2017, pages 5998-6008, arxiv.org/abs/1706.03762.